

AIAC-ASSIGNMENT 6.3

Name :E.ANJANI

HT. No : 2303A52419

Batch: 32

Q1. Task: Prompt AI to generate a function that displays all Automorphic numbers between 1 and 1000 using a for loop.

```
Task1.py > ...
1  """
2  1.wrt a python function that genartes automorphic numbers within range from 1 to 1000. using for loop
3  """
4  import time
5
6
7  def is_automorphic(num):
8      square = num ** 2
9      return str(square).endswith(str(num))
10 def generate_automorphic_numbers(start, end):
11     automorphic_numbers = []
12     for num in range(start, end + 1):
13         if is_automorphic(num):
14             automorphic_numbers.append(num)
15     return automorphic_numbers
16 t1 = time.time()
17 automorphic_numbers = generate_automorphic_numbers(1, 1000)
18 print("Time taken using for loop:", time.time() - t1)
19 print("Automorphic numbers between 1 and 1000 are:", automorphic_numbers)
20
```

```
...
2.write a python function that genartes automorphic numbers within range from 1 to 1000. using while loop
...
def generate_automorphic_numbers_while(start, end):
    automorphic_numbers = []
    num = start
    while num <= end:
        if is_automorphic(num):
            automorphic_numbers.append(num)
        num += 1
    return automorphic_numbers
t2 = time.time()
automorphic_numbers_while = generate_automorphic_numbers_while(1, 1000)
print("Time taken using while loop:", time.time() - t2)
print("Automorphic numbers between 1 and 1000 using while loop are:", automorphic_numbers_while)
```

```
PS D:\Course\AIAC\28-1-2026> & C:/Python314/python.exe d:/Course/AIAC/28-1-2026/Task1.py
Time taken using for loop: 0.0004050731658935547
Automorphic numbers between 1 and 1000 are: [1, 5, 6, 25, 76, 376, 625]
Time taken using while loop: 0.0004239082336425781
Automorphic numbers between 1 and 1000 using while loop are: [1, 5, 6, 25, 76, 376, 625]
PS D:\Course\AIAC\28-1-2026> █
```

Explanation:

For loop is taking less time then while loop. Use for loop when you know when to stop if not use while loop.

Q2. Task: Ask AI to write nested if-elif-else conditions to classify online shopping feedback as Positive, Neutral, or Negative based on a numerical rating (1–5).

```
Task2.py > classify_feedback_dict
1 import time
2 #write a python functuion that classify online shopping feedback into positive, negative and neutral based
  on numerical rating (1-5) using nested if-elif-else statements.
3 def classify_feedback(rating):
4     if rating >= 1 and rating <= 5:
5         if rating >= 4:
6             return "Positive"
7         elif rating == 3:
8             return "Neutral"
9         else:
10            return "Negative"
11    else:
12        return "Invalid rating. Please provide a rating between 1 and 5."
13 # Example usage:
14 t1 = time.time()
15 print(classify_feedback(5))
16 print("Time taken usning if-elif-else statements:", time.time() - t1) # Output: Positive
17 print(classify_feedback(3)) # Output: Neutral
18 print(classify_feedback(1)) # Output: Negative
19 print(classify_feedback(6)) # Output: Invalid rating. Please provide a rating between 1 and 5.
20
21
22
23
24
```

```
print("\n")
#write a python functuion that classify online shopping feedback into positive, negative and neutral based
on numerical rating (1-5) using dictionary or match case.
def classify_feedback_dict(rating):
    feedback_dict = {
        5: "Positive",
        4: "Positive",
        3: "Neutral",
        2: "Negative",
        1: "Negative"
    }
    return feedback_dict.get(rating, "Invalid rating. Please provide a rating between 1 and 5.")
# Example usage:
t2 = time.time()
print(classify_feedback_dict(5))
print("Time taken using dictionary or match case:", time.time() - t2) # Output: Positive
print(classify_feedback_dict(3)) # Output: Neutral
print(classify_feedback_dict(1)) # Output: Negative
print(classify_feedback_dict(6)) # Output: Invalid rating. Please provide a rating between 1 and 5.
```

```

PS D:\Course\AIAC\28-1-2026> & C:/Python314/python.exe d:/Course/AIAC/28-1-2026/Task2.py
● Positive
Time taken using if-elif-else statements: 0.00010204315185546875
Neutral
Negative
Invalid rating. Please provide a rating between 1 and 5.

Positive
Time taken using dictionary or match case: 3.743171691894531e-05
Neutral
Negative
Invalid rating. Please provide a rating between 1 and 5.
○ PS D:\Course\AIAC\28-1-2026>

```

Explanation:

Condition ensures correctness of the code, Dictionary improves readability

Q3. Task 3: Statistical_operations

Define a function named `statistical_operations(tuple_num)` that performs the following statistical operations on a tuple of numbers:

- Minimum, Maximum
- Mean, Median, Mode
- Variance, Standard Deviation

While writing the function, observe the code suggestions provided by GitHub Copilot. Make decisions to accept, reject, or modify the suggestions based on their relevance and correctness

```
Task3.py • Task2.py
28-1-2026 > Task3.py > ...
1  """
2  1. write a python function nsame statistical_ooperation(tuple_numm) and perform Minimum, maximum, mean,
3  | median, mode, variance and standard deviation.
4  """
5  import statistics
6  def statistical_operation(tuple_numm):
7      if len(tuple_numm) == 0:
8          return "The tuple is empty. Please provide a tuple with numerical values."
9
10     minimum = min(tuple_numm)
11     maximum = max(tuple_numm)
12     mean = statistics.mean(tuple_numm)
13     median = statistics.median(tuple_numm)
14     try:
15         mode = statistics.mode(tuple_numm)
16     except statistics.StatisticsError:
17         mode = "No unique mode found"
18     variance = statistics.variance(tuple_numm)
19     std_deviation = statistics.stdev(tuple_numm)
20
21     return {
22         "Minimum": minimum,
23         "Maximum": maximum,
24         "Mean": mean,
25         "Median": median,
26         "Mode": mode,
27         "Variance": variance,
28         "Standard Deviation": std_deviation
29     }
30 # Example usage:
31 data = (1, 2, 2, 3, 4, 5, 5, 5)
32 result = statistical_operation(data)
33 for key, value in result.items():
34     print(f"{key}: {value}")
35
PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/Activate.ps1
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe d:/Course/AIAC/28-1-2026/Task3.py
Minimum: 1
Maximum: 5
Mean: 3.375
Median: 3.5
Mode: 5
Variance: 2.5535714285714284
Standard Deviation: 1.5979898086569353
(AIAC_env) PS D:\Course\AIAC>
```

Q4. Task 4: Teacher Profile

- **Prompt:** Create a class Teacher with attributes teacher_id, name, subject, and experience. Add a method to display teacher details.
- **Expected Output:** Class with initializer, method, and object creation.

```
Task4.py X
28-1-2026 > Task4.py > ...
1  '''
2  create a class with Teacher with attributes teacher_id, name, subject, and experience
3  Method to display teacher details
4  '''
5  class Teacher:
6      def __init__(self, teacher_id, name, subject, experience):
7          self.teacher_id = teacher_id
8          self.name = name
9          self.subject = subject
10         self.experience = experience
11
12         def display_details(self):
13             print(f"Teacher ID: {self.teacher_id}")
14             print(f"Name: {self.name}")
15             print(f"Subject: {self.subject}")
16             print(f"Experience: {self.experience} years")
17     # Example usage
18     teacher1 = Teacher(1, "Alice Smith", "Mathematics", 10)
19     teacher1.display_details()
20     teacher2 = Teacher(2, "Bob Johnson", "Science", 8)
21     teacher2.display_details()
22     teacher3 = Teacher(3, "Charlie Brown", "History", 5)
23     teacher3.display_details()
24     teacher4 = Teacher(4, "Diana Prince", "English", 12)
25     teacher4.display_details()
```

```
PROBLEMS OUTPUT PORTS DEBUG CONSOLE TERMINAL
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe d:/Course/AIAC/28-1-2026/Task4.py
● Teacher ID: 1
  Name: Alice Smith
  Subject: Mathematics
  Experience: 10 years
  Teacher ID: 2
  Name: Bob Johnson
  Subject: Science
  Experience: 8 years
  Teacher ID: 3
  Name: Charlie Brown
  Subject: History
  Experience: 5 years
  Teacher ID: 4
  Name: Diana Prince
  Subject: English
  Experience: 12 years
○ (AIAC_env) PS D:\Course\AIAC> 
```

Q5. Task #5 – Zero-Shot Prompting with Conditional Validation

Use zero-shot prompting to instruct an AI tool to generate a function that validates an Indian mobile number.

Requirements

- The function must ensure the mobile number:

- o Starts with 6, 7, 8, or 9

- o Contains exactly 10 digits

```
Task5.py X
28-1-2026 > Task5.py > ...
1  '''
2  write a python function name vaild indian number and check number starts with 6 7 8 9
3  and length is 10 digit
4  '''
5  def vaild_indian_number(number):
6      if len(number) == 10 and number[0] in '6789' and number.isdigit():
7          return True
8      return False
9  print(vaild_indian_number("9876543210")) # True
10 print(vaild_indian_number("1234567890")) # False
11

(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe d:/Course/AIAC/28-1-2026/Task5.py
True
False
(AIAC_env) PS D:\Course\AIAC>
```

Q6. Task Description #6 (Loops – Armstrong Numbers in a Range)

Task: Write a function using AI that finds all Armstrong numbers in a user-specified range (e.g., 1 to 1000).

28-1-2026 > Task6.py > ...

```
1  '''
2  write a python function that finds armstrong numbers from 1 to 1000
3  using for loop and digit power logic.
4  '''
5  def find_armstrong_numbers():
6      armstrong_numbers = []
7      for num in range(1, 1001):
8          # Convert number to string to easily iterate over digits
9          digits = str(num)
10         num_digits = len(digits)
11         sum_of_powers = sum(int(digit) ** num_digits for digit in digits)
12
13         if sum_of_powers == num:
14             armstrong_numbers.append(num)
15
16     return armstrong_numbers
17 # Example usage
18 armstrong_numbers = find_armstrong_numbers()
19 print("Armstrong numbers from 1 to 1000:", armstrong_numbers)
```

```
'''
write a python function that finds armstrong numbers from 1 to 1000
using for loop and digit power logic. optimized version using list comprehension
'''
def find_armstrong_numbers_optimized():
    return [num for num in range(1, 1001) if sum(int(digit) ** len(str(num)) for digit in str(num)) == num]
# Example usage
armstrong_numbers_optimized = find_armstrong_numbers_optimized()
print("Armstrong numbers from 1 to 1000 (optimized):", armstrong_numbers_optimized)
```

Output:

```
PROBLEMS  OUTPUT  PORTS  DEBUG CONSOLE  TERMINAL
PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/Activate.ps1
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe d:/Course/AIAC/28-1-2026/Task6.py
Armstrong numbers from 1 to 1000: [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
Armstrong numbers from 1 to 1000 (optimized): [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
(AIAC_env) PS D:\Course\AIAC>
```

Explanation:

Using for loop it is easy to understand and debug the code best for bigginer.

List comprehension is minimal and hard to debug

Q7. Task Description #7 (Loops – Happy Numbers in a Range)

Task: Generate a function using AI that displays all Happy Numbers within a user-specified range (e.g., 1 to 500).

```
2  write a python function to display all happnumbers in rnage 1 to 500
3  '''
4  import time
5
6
7  def is_happy_number(n):
8      seen = set()
9      while n != 1 and n not in seen:
10         seen.add(n)
11         n = sum(int(digit) ** 2 for digit in str(n))
12     return n == 1
13 def happy_numbers_in_range(start, end):
14     happy_numbers = []
15     for num in range(start, end + 1):
16         if is_happy_number(num):
17             happy_numbers.append(num)
18     return happy_numbers
19 t1 = time.time()
20 happy_numbers = happy_numbers_in_range(1, 500)
21 t2 = time.time()
22 print("Time taken without optimization:", t2 - t1)
23 print("Happy numbers between 1 and 500 are:", happy_numbers)
```

```
...
write a python function to display all happnumbers in rnage 1 to 500
Optimize the above code using cycle detection
...
import time
def is_happy_number_optimized(n):
    def get_next(number):
        return sum(int(digit) ** 2 for digit in str(number))

    slow = n
    fast = get_next(n)
    while fast != 1 and slow != fast:
        slow = get_next(slow)
        fast = get_next(get_next(fast))
    return fast == 1
def happy_numbers_in_range_optimized(start, end):
    happy_numbers = []
    for num in range(start, end + 1):
        if is_happy_number_optimized(num):
            happy_numbers.append(num)
    return happy_numbers
t1 = time.time()
happy_numbers_optimized = happy_numbers_in_range_optimized(1, 500)
t2 = time.time()
print("Time taken with optimization:", t2 - t1)
print("Happy numbers between 1 and 500 are (optimized):", happy_numbers_optimized)
```


Output:

```
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe d:/Course/AIAC/28-1-2026/Task7.py
Time taken without optimization: 0.0021097660064697266
Happy numbers between 1 and 500 are: [1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91, 94, 97, 100, 103, 109, 129, 130, 133,
139, 167, 176, 188, 190, 192, 193, 203, 208, 219, 226, 230, 236, 239, 262, 263, 280, 291, 293, 301, 302, 310, 313, 319, 320, 326, 329, 331,
338, 356, 362, 365, 367, 368, 376, 379, 383, 386, 391, 392, 397, 404, 409, 440, 446, 464, 469, 478, 487, 490, 496]
Time taken with optimization: 0.003709077835083008
Happy numbers between 1 and 500 are (optimized): [1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91, 94, 97, 100, 103, 109, 12
9, 130, 133, 139, 167, 176, 188, 190, 192, 193, 203, 208, 219, 226, 230, 236, 239, 262, 263, 280, 291, 293, 301, 302, 310, 313, 319, 320, 326
, 329, 331, 338, 356, 362, 365, 367, 368, 376, 379, 383, 386, 391, 392, 397, 404, 409, 440, 446, 464, 469, 478, 487, 490, 496]
(AIAC_env) PS D:\Course\AIAC>
```

Explanation:

The optimized cycle detection method is the best choice because it reduces memory usage, improves efficiency, and scales better while still producing correct results. The set-based approach is easier for beginners, but the optimized method is more professional and performance-oriented.

Q8. Task Description #8 (Loops – Strong Numbers in a Range)

Task: Generate a function using AI that displays all Strong Numbers (sum of factorial of digits equals the number, e.g., $145 = 1! + 4! + 5!$) within a given range.

```
task8.py x
28-1-2026 > Task8.py > ...
1  """
2  write a python function to display strong numbers with given range.
3  """
4  import time
5
6
7  def factorial(n):
8      if n == 0 or n == 1:
9          return 1
10     else:
11         return n * factorial(n - 1)
12
13 def is_strong_number(num):
14     sum_of_factorials = 0
15     temp = num
16     while temp > 0:
17         digit = temp % 10
18         sum_of_factorials += factorial(digit)
19         temp //= 10
20     return sum_of_factorials == num
21
22 def display_strong_numbers(start, end):
23     strong_numbers = []
24     for num in range(start, end + 1):
25         if is_strong_number(num):
26             strong_numbers.append(num)
27     return strong_numbers
28
29 # Example usage:
30 start_range = 1
31 end_range = 500
32 t1=time.time()
33 strong_numbers_in_range = display_strong_numbers(start_range, end_range)
34 t2=time.time()
35 print(f"Time taken: {t2-t1} seconds")
36 print(f"Strong numbers between {start_range} and {end_range}: {strong_numbers_in_range}")
37
```

```

35
36 '''
37 write a python function to display strong numbers with given range.
38 optimized version precomputing factorials.
39 '''
40
41 import time
42 def precompute_factorials():
43     factorials = {}
44     for i in range(10):
45         factorials[i] = factorial(i)
46     return factorials
47 def is_strong_number_optimized(num, factorials):
48     sum_of_factorials = 0
49     temp = num
50     while temp > 0:
51         digit = temp % 10
52         sum_of_factorials += factorials[digit]
53         temp //= 10
54     return sum_of_factorials == num
55 def display_strong_numbers_optimized(start, end):
56     factorials = precompute_factorials()
57     strong_numbers = []
58     for num in range(start, end + 1):
59         if is_strong_number_optimized(num, factorials):
60             strong_numbers.append(num)
61     return strong_numbers
62 # Example usage:
63 start_range = 1
64 end_range = 500
65 t1=time.time()
66 strong_numbers_in_range_optimized = display_strong_numbers_optimized(start_range, end_range)
67 t2=time.time()
68 print(f"Time taken (optimized): {t2-t1} seconds")
69 print(f"Strong numbers between {start_range} and {end_range} (optimized): {strong_numbers_in_range_optimized}")
70

```

Output:

```

PROBLEMS  OUTPUT  PORTS  DEBUG CONSOLE  TERMINAL

PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/Activate.ps1
● (AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe d:/Course/AIAC/28-1-2026/Task8.py
● Time taken: 0.0006704330444335938 seconds
  Strong numbers between 1 and 500: [1, 2, 145]
  Time taken (optimized): 0.00022077560424804688 seconds
  Strong numbers between 1 and 500 (optimized): [1, 2, 145]
○ (AIAC_env) PS D:\Course\AIAC>

```

Explanation:

The basic approach identifies strong numbers correctly but recalculates factorials repeatedly, leading to unnecessary computation.

The optimized method precomputes factorials of digits (0–9), reducing repeated calculations and improving efficiency.

Hence, the optimized approach is more suitable for larger ranges due to better performance and scalability.

Q9. Task #9 – Few-Shot Prompting for Nested Dictionary Extraction

Objective

Use few-shot prompting (2–3 examples) to instruct the AI to create a function that parses a nested dictionary representing student information.

```
201 2020 7 tasks.py 7 ...
1  '''
2  write a python function that parses a nested dictionary of student
   information Requirements:
3  Full Name
4  o Branch
5  o SGPA
6
7  example input:
8  | "full_name": "Alice Johnson",
9  |         | "branch": "Computer Science",
10 |         | "sgpa": 8.5
11 |         | full_name": "Bob Smith",
12 |         | "branch": "Mechanical Engineering",
13 |         | "sgpa": 7.8
14  '''
```

```

15 def parse_student_info(students):
16     parsed_info = []
17     for student_id, info in students.items():
18         student_data = {
19             "Full Name": info.get("full_name"),
20             "Branch": info.get("branch"),
21             "SGPA": info.get("sgpa")
22         }
23         parsed_info.append(student_data)
24     return parsed_info
25 # Example usage
26 students = {
27     "student_1": {
28         "full_name": "Alice Johnson",
29         "branch": "Computer Science",
30         "sgpa": 8.5
31     },
32     "student_2": {
33         "full_name": "Bob Smith",
34         "branch": "Mechanical Engineering",
35         "sgpa": 7.8
36     },
37     "student_3": {
38         "full_name": "Charlie Brown",
39         "branch": "Electrical Engineering",
40         "sgpa": 9.1
41     }
42 }
43 parsed_students = parse_student_info(students)
44 for student in parsed_students:
45     print(student)
46

```

Output:

```

PROBLEMS  OUTPUT  PORTS  DEBUG CONSOLE  TERMINAL

(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe d:/Course/AIAC/28-1-2026/Task9.py
• {'Full Name': 'Alice Johnson', 'Branch': 'Computer Science', 'SGPA': 8.5}
  {'Full Name': 'Bob Smith', 'Branch': 'Mechanical Engineering', 'SGPA': 7.8}
  {'Full Name': 'Charlie Brown', 'Branch': 'Electrical Engineering', 'SGPA': 9.1}
○ (AIAC_env) PS D:\Course\AIAC>

```

Q10. Task Description #10 (Loops – Perfect Numbers in a Range)

Task: Generate a function using AI that displays all Perfect Numbers within a user-specified range (e.g., 1 to 1000).

```

28-1-2026 > Task10.py > ...
1  '''
2  write a python funcyion that displays all perfect numbers in range from 1 to 1000
3  '''
4  import time
5  def is_perfect_number(n):
6      sum_of_divisors = 0
7      for i in range(1, n):
8          if n % i == 0:
9              sum_of_divisors += i
10     return sum_of_divisors == n
11 def display_perfect_numbers():
12     perfect_numbers = []
13     for num in range(1, 1001):
14         if is_perfect_number(num):
15             perfect_numbers.append(num)
16     return perfect_numbers
17 if __name__ == "__main__":
18     t1 = time.time()
19     perfect_numbers = display_perfect_numbers()
20     t2 = time.time()
21     print("Time taken to find perfect numbers:", t2 - t1, "seconds")
22     print("Perfect numbers between 1 and 1000 are:", perfect_numbers)
23

```

```

28-1-2026 > Task10.py > ...
27  '''
28  write a python funcyion that displays all perfect numbers in range from 1 to 1000 optimized using divisor check only up to square root of n
29  '''
30  import time
31  import math
32  def is_perfect_number_optimized(n):
33      if n < 2:
34          return False
35      sum_of_divisors = 1 # 1 is a divisor of all n > 1
36      for i in range(2, int(math.sqrt(n)) + 1):
37          if n % i == 0:
38              sum_of_divisors += i
39              if i != n // i:
40                  sum_of_divisors += n // i
41      return sum_of_divisors == n
42  def display_perfect_numbers_optimized():
43      perfect_numbers = []
44      for num in range(1, 1001):
45          if is_perfect_number_optimized(num):
46              perfect_numbers.append(num)
47      return perfect_numbers
48  if __name__ == "__main__":
49      t1 = time.time()
50      perfect_numbers_optimized = display_perfect_numbers_optimized()
51      t2 = time.time()
52      print("Time taken to find perfect numbers (optimized):", t2 - t1, "seconds")
53      print("Perfect numbers between 1 and 1000 (optimized) are:", perfect_numbers_optimized)

```

Output:

```

PROBLEMS  OUTPUT  PORTS  DEBUG CONSOLE  TERMINAL

PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/Activate.ps1
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe d:/Course/AIAC/28-1-2026/Task10.py
Time taken to find perfect numbers: 0.021500110626220703 seconds
Perfect numbers between 1 and 1000 are: [6, 28, 496]
Time taken to find perfect numbers (optimized): 0.001857757568359375 seconds
Perfect numbers between 1 and 1000 (optimized) are: [6, 28, 496]
(AIAC_env) PS D:\Course\AIAC>

```

Explanation:

The basic method checks all possible divisors up to the number, resulting in higher time complexity.

The optimized approach checks divisors only up to the square root of the number, significantly improving efficiency while producing the same correct results.